

ADO-01

ENUNCIADO – I - Faça uma classe java contendo dois vetores ou crie um objeto java com dois atributos. A ideia é cadastrar uma lista de nomes e uma lista de senhas numéricas, dados que deverão ser digitados via teclado.

Use a Classe JOptionPane para leitura e exibição dos dados. A classe deverá conter um menu de controle -Tamanho de 5 elementos cada. Opções de menu - 1 ler nome e senha numérica 2 listar nomes e senhas (lado a lado) 3 buscar um nome de forma sequencial 4 buscar uma senha de forma sequencial (trazer a posição caso encontre) 5 classificar por nome usando bubble sort 6 classificar as senhas usando bubble sort. 7 Finalizar.

II - Responda as questões a seguir

- 1 Defina o que você entendeu sobre comportamento assintótico de algoritmos
- 2 Em que consiste a notação Big O.
- 3 Em que consiste a notação Big Omega
- 4 Em que consiste a notação Big Teta.
- 5 Defina uma função constante.
- 6 Defina uma função linear.
- 7 Defina uma função quadrática / exponencial.

NOTA: Ado é uma atividade para ser entregue em grupo, todo mundo faz mas apenas um dos membros do grupo entrega a versão melhor.

CÓDIGO:

Usuario.java (classe de dados para o nome e senha)

```
public class Usuario {  
    private String nome;  
    private int senha;  
  
    Usuario(String nome, int senha) {  
        this.nome = nome;  
        this.senha = senha;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
    public int getSenha() {  
        return senha;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

```

    }
    public void setSenha(int senha) {
        this.senha = senha;
    }
}

```

Principal.java

```

import javax.swing.*;
import java.util.Scanner;

public class Principal {
    static Usuario[] usuarios = new Usuario[5];
    static int ultimaPosicao = -1;

    public static void main(String[] args) {
        String txtMenu = "1. ler nome e senha numérica\n";
        txtMenu += "2. listar nomes e senhas (lado a lado)\n";
        txtMenu += "3. buscar um nome de forma sequencial \n";
        txtMenu += "4. buscar uma senha de forma sequencial\n";
        (trazer a posição caso encontre) \n";
        txtMenu += "5. classificar por nome usando bubble sort \n";
        txtMenu += "6. classificar as senhas usando bubble sort \n";
        txtMenu += "7. finalizar \n\n\n";

        boolean executando = true;

        do{
            int opcao = Integer.parseInt(JOptionPane.showInputDialog(null,
txtMenu, "MENU INICIAL", JOptionPane.PLAIN_MESSAGE));

            switch(opcao) {
                case 1:
                    cadastrar();
                    break;
                case 2:
                    listar(usuarios);
                    break;
                case 3:
                    buscarNome();
                    break;
                case 4:
                    buscarSenha();
                    break;
                case 5:
                    listarOrdenadoNome();
                    break;
            }
        } while (executando);
    }
}

```

```

        case 6:
            listarOrdenadoSenha();
            break;
        case 7:
            executando = false;
    }
} while(executando);

JOptionPane.showMessageDialog(null, "Sistema encerrado");
System.exit(0);
}

private static void buscarNome() {
    String nome = JOptionPane.showInputDialog("Digite o nome
desejado: ");

    Usuario usuarioDesejado = null;

    for(Usuario u : usuarios) {
        if(u != null && u.getNome().equalsIgnoreCase(nome)) {
            usuarioDesejado = u;
            break;
        }
    }

    if(usuarioDesejado == null) {
        JOptionPane.showMessageDialog(null, "Usuário com nome " +
nome + " não encontrado!");
    } else {
        JOptionPane.showMessageDialog(null, "Nome: %s | Senha: %d\
n".formatted(usuarioDesejado.getNome(),
usuarioDesejado.getSenha()));
    }
}

private static void buscarSenha() {
    int senha = Integer.parseInt(JOptionPane.showInputDialog("Digite
a senha desejada: "));

    Usuario usuarioDesejado = null;

    for(Usuario u : usuarios) {
        if(u != null && u.getSenha() == senha) {
            usuarioDesejado = u;
            break;
        }
    }
}

```

```

    }

    if(usuarioDesejado == null) {
        JOptionPane.showMessageDialog(null, "Usuário com a senha "
+ senha + " não encontrado!");
    }else {
        JOptionPane.showMessageDialog(null, "Nome: %s | Senha: %d\
n".formatted(usuarioDesejado.getNome(),
usuarioDesejado.getSenha()));
    }
}

public static void cadastrar() {
    if(ultimaPosicao == usuarios.length - 1) {
        JOptionPane.showMessageDialog(null, "Cadastros já
realizados!");
        return;
    }

    int opcao = 0;

    do {
        int i = ultimaPosicao + 1;

        String nome = JOptionPane.showInputDialog("Digite o nome do
" + (i+1) + "º usuário: ").toLowerCase();
        nome = nome.substring(0, 1).toUpperCase() +
nome.substring(1); //capitalize name

        int senha =
Integer.parseInt(JOptionPane.showInputDialog("Digite a senha do " +
(i+1) + "º usuário: "));

        Usuario usuario = new Usuario(nome, senha);

        usuarios[i] = usuario;
        ultimaPosicao = i;

        if(ultimaPosicao < usuarios.length - 1) {
            String resposta = JOptionPane.showInputDialog("Deseja
cadastrar outro usuário? [S/N] ");
            opcao = resposta.equalsIgnoreCase("S") ? 1 : 0;
        }else {
            opcao = 0;
        }
    }
}

```

```

    }while(opcao == 1);

}

public static void listar(Usuario[] listaUsuarios) {
    String saida = "\n\n";

    //enhanced for
    for(Usuario usuario : listaUsuarios) {
        if(usuario != null) {
            saida += "Nome: %s | Senha: %d\n"
n".formatted(usuario.getNome(), usuario.getSenha());
        }
    }

    JOptionPane.showMessageDialog(null, saida);
}

public static void listarOrdenadoNome() {
    Usuario[] lista = bubbleSortNome(usuarios);

    listar(lista);
}

public static void listarOrdenadoSenha() {
    Usuario[] lista = bubbleSortSenha(usuarios);

    listar(lista);
}

public static Usuario[] bubbleSortSenha(Usuario[] vetor) {
    Usuario[] mVetor = vetor.clone();

    boolean troca = true;
    while(troca) {
        troca = false;
        for(int i = 0; i < ultimaPosicao; i++) {
            if(mVetor[i].getSenha() > mVetor[i+1].getSenha()) {
                trocar(mVetor, i, i+1);
                troca = true;
            }
        }
    }

    return mVetor;
}

```

```

public static Usuario[] bubbleSortNome(Usuario[] vetor) {
    Usuario[] mVetor = vetor.clone();

    boolean troca = true;
    while(troca) {
        troca = false;
        for(int i = 0; i < ultimaPosicao; i++) {
            if(ehMaior(mVetor[i].getNome(), mVetor[i+1].getNome())) {
                trocar(mVetor, i, i+1);
                troca = true;
            }
        }
    }

    return mVetor;
}

//devolve 'true' apenas se str1 é maior
public static boolean ehMaior(String str1, String str2) {
    for(int i = 0; i < str1.length(); i++) {
        char c1 = str1.charAt(i);
        char c2;

        try {
            c2 = str2.charAt(i);
        } catch (IndexOutOfBoundsException e) { //a str2 é menor. ex:
str1 = "abc" e str2 = "ab"
            return true;
        }

        if(c1 > c2) {
            return true;
        } else if(c2 > c1) {
            return false;
        }
    }

    return false; //as strings são iguais
}

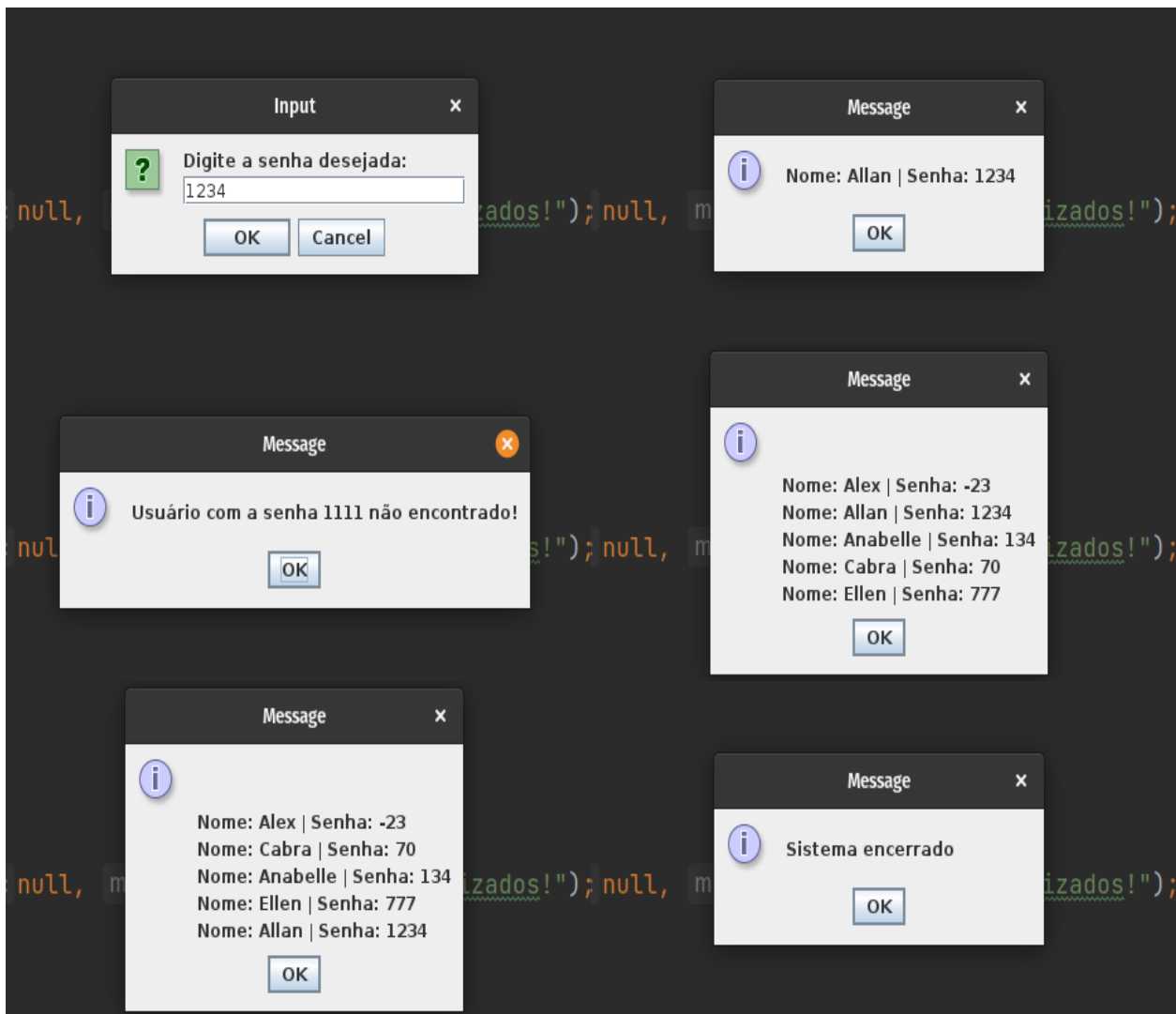
private static void trocar(Usuario[] vetor, int i1, int i2) {
    Usuario tmp = vetor[i1];
    vetor[i1] = vetor[i2];
    vetor[i2] = tmp;
}

```

```
}
```

PRINTS DE EXECUÇÃO:





1. Defina o que você entendeu sobre comportamento assintótico de algoritmos

Serve para medir a eficiência com relação ao tempo de execução ou uso de memória de um algoritmo.

2. Em que consiste a notação Big O.

Serve para indicar a eficiência no PIOR CENÁRIO POSSÍVEL.

3. Em que consiste a notação Big Omega

Serve para indicar a eficiência no MELHOR CENÁRIO POSSÍVEL.

4. Em que consiste a notação Big Teta.

Serve para indicar a eficiência no limite exato, entre o limite superior e inferior

5. Defina uma função constante.

$f(n) = c$, onde n = quantidade de interações e c = custo ou tempo.

Para qualquer valor de n , o tempo é o mesmo. Forma uma linha reta e constante no gráfico.

6. Defina uma função linear.

$f(n) = a.n$, onde n = quantidade de interações.

O custo de tempo aumenta de forma proporcional à entrada. Forma uma reta crescente no gráfico.

7. Defina uma função quadrática / exponencial.

$f(n) = a^n$, em que “a” é um número real dado, $a > 0$ e $a \neq 1$ e n = quantidade de interações.

O custo de tempo cresce de forma exponencial ao número de entrada. Forma uma curva exponencial crescente no gráfico.